

Improving Automated Reasoning for First-Order Logic Using Optimised Proof Search

Deepansh Gulati

Griffith University, Gold Coast, Australia

Abstract.

In this research we designed an Automated Reasoning System for First-Order Logic (FOL) using the Sequent Calculus Proof Search method. We have also built a parser to transform FOL expressions from concrete representation (string) to Abstract Syntax Tree (AST) and then to sequents form for evaluation. We have created a Baseline Solver that uses Algorithm 2 as its core algorithm. Using Algorithm 2 we implement a Recursive Proof Search to evaluate the logical rules of our given problem. We propose an Improved Solver to improve the Baseline Solver by introducing Repeated-State Caching and Coordinated Quantifier Instantiation. Our research was tested over several different sets of data representing various levels of difficulty. Results clearly indicate the Improved Solver can prove more formulas than the Baseline Solver especially for medium-hard problems with only slightly increased Runtime.

Keywords: automated reasoning, first-order logic, sequent calculus, proof search, theorem proving

1 Introduction

Automated Reasoning is one of the most critical disciplines within Computer Science, focusing on how to enable machines to execute logical inference automatically. As such, Automated Reasoning has many applications across various domains including formal verification (the ability to formally prove the correctness of systems), artificial intelligence and program analysis.

First-Order Logic is a very powerful framework used by Automated Reasoning systems to describe complex relationships with quantification and predicates. This makes First-Order Logic a key formalism for Automated Reasoning systems.

An important methodology for Automated Reasoning is proof search using formal proof systems like sequent calculus. Using this methodology, formulas expressed in First-Order Logic are converted to sequences of the form $\Gamma \vdash \Delta$ where Γ represents assumptions made prior to the application of the rules of inference and Δ represents conclusions derived through the application of those same rules of inference. Once a closed sequence is derived, it is inferred that the formula is valid.

Proof Search in First-Order Logic is difficult because of the size of the search space and the presence of quantifiers that greatly increase computational complexity.

In this project, an Automated Reasoning system was developed that utilizes the sequent calculus framework. A parser was developed to parse First-Order Logic Formulas and convert them to internal representation. Following the textbook's Algorithm 2 [1] a Baseline Solver was developed that performs recursive proof search utilizing sequent rules. An Improved Solver was also proposed to improve upon the performance of the baseline method.

The main contribution of this work is the design and evaluation of an improved proof-search strategy. The improved solver uses repeated-state caching and coordinated quantifier instantiation, which enables the improved solver to more effectively handle more complex formulas than the baseline solver. The system was tested on multiple datasets of varying difficulty and the results showed that the improved approach enables the system to prove many more formulas than the baseline solver, especially for medium and hard cases.

2 Proposed Approach

This project implements an automated reasoning pipeline for first order logic formulae using a proof search method based upon sequent calculus.

Input (in the form of formulas) are parsed into an abstract syntax tree and converted to an initial sequent. These initial sequent were attempted to be proven by both the baseline solver and improved solver.

Baseline solver - the baseline uses Algorithm 2's structure [1] to perform proof-search using the sequent calculus rules. Once a closed-sequent is found, or no rule can be applied, the proof-search terminates. A sequent is represented as $\Gamma \vdash \delta$, where Γ contains assumptions on the left side and δ contains conclusions on the right side. The proof succeeds when both sides of the sequent contain the same formula. The identity-rule is used here.

Parser - parser supports main constructs for first order logic from this course including predicates, constants, variables, negation, conjunction, disjunction, implication, universal quantification, existential quantification. All formulas are represented using abstract syntax trees. Logical rules are applied structurally.

Improved solver - the improved algorithm extends the baseline by adding repeated-state caching and more coordinated quantifier instantiation. Repeated-state caching prevents the solver from visiting the same sequent multiple times. Thus, reducing redundant paths in the proof-search process. Quantifier improvement is designed to handle cases where universal formulae on both sides of a sequent need to be instantiated with the

same constant. This helps the solver prove more difficult formulae that include nested quantifiers and chains of implications. This improvement reduces the branching factor of the proof search and avoids redundant exploration of equivalent sequents, which explains the increased success rate on complex formulas.

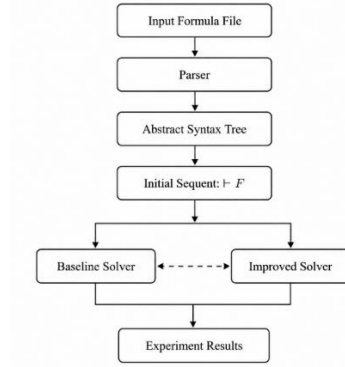


Figure 1. Workflow of the implemented automated reasoning system

3 Implementation and Experiment

All experiments were conducted on a system with an Intel Core i9-13900HX processor, 32 GB RAM, and an NVIDIA GeForce RTX 4080 GPU, running Windows 11. The implementation was executed locally using Python 3.11, and runtime measurements were recorded during execution.

3.1 Dataset Design

The manual evaluation datasets were developed for testing the solver’s ability to process formulas with increasing complexity. These three data sets (Easy, Medium, and Hard) each contain five first order logic formulas. The Easy dataset has relatively simply constructed formulas. They have only minimal structural characteristics, in most cases only a single step is needed to establish the truth of the formula. The Medium data set has formulas which include implication statements, conjunction and disjunction statements and quantification statements requiring two or more steps in the search for proofs. The Hard data set includes nested quantifications and requires coordination among quantifier instantiations on both sides of the sequent. This format was used to evaluate both the correctness and scalability of the improved proof-search strategy. Correctness is confirmed by using the Easy data set as an example of how the baseline rules are properly implemented. Scalability of the improved solver is tested by using the Medium and Hard datasets, which provide a wider variety of search space sizes and complexities within quantifier interaction rather than a toy example.

Table 1. Baseline solver vs improved solver per dataset

Dataset	Formula type tested	Purpose
Easy	Simple FOL formulae and basic quantifiers	Tests basic correctness
Medium	Implications and mixed connectives	Tests multi-step proof search
Hard	Nested quantifiers and quantifier chains	Tests complex proof-search behaviour

Example formulas from each dataset are shown below:

Easy:

1. $\forall x P(x) \rightarrow P(a)$
2. $P(a) \rightarrow P(a)$

Medium:

1. $\forall x (P(x) \rightarrow Q(x)) \wedge P(a) \rightarrow Q(a)$
2. $\forall x P(x) \rightarrow \exists x P(x)$

Hard:

1. $\forall x (P(x) \rightarrow \exists y Q(x, y)) \rightarrow \exists y \forall x Q(x, y)$
2. $\forall x \exists y R(x, y) \rightarrow \exists y \forall x R(x, y)$

3.2 Results table

Table 2. Baseline Solver VS Improved Solver per Dataset

Dataset	# Formulas	Baseline Solved	Improved Solved	Baseline time (s)	Improved time (s)
Easy	5	5	5	0.000101	0.000236
Medium	5	2	4	0.000231	0.000719
Hard	5	1	3	0.000245	0.000959

The improved algorithm solves more formulas than the baseline, especially in medium and hard datasets. The baseline struggles with formulas that require coordinated handling of quantifiers. The improved solver performs better due to caching and better handling of quantifier instantiation. The improved method takes slightly more time due to additional checks, but the increase is minimal.

Although the improved solver is slightly slower, it solved 4/5 medium formulas compared with 2/5 for the baseline, and 3/5 hard formulas compared with 1/5 for the baseline. This indicates that the improvement is more effective for difficult formulas involving quantifier interaction, even though it adds minor runtime overhead.

We implemented our solution using a modular approach in Python [2] data, with distinct modules responsible for a. parsing First Order Logic (FOL) formulas from text files to Abstract Syntax Trees (ASTs), b. constructing and evaluating sequent during Proof Search; and c. performing evaluations.

We implemented the basic solver according to the algorithmic structure described in Algorithm 2 [1]. In particular, we employed recursive proof search on sequent. The solver applies the logical rules as needed based on the formula type encountered: negation, conjunction, disjunction, implication, universal/existential quantification. The solver considers a sequent proven when it finds a formula appearing on both sides of the sequent.

The improved solver builds upon the basic solver's architecture by adding two additional features: a. repeated-state caching (to avoid unnecessary recomputation); and b. coordinated quantifier instantiation (for handling nested quantifiers). When an existential/universal quantifier appears on both sides of a sequent, the improved solver will instantiate these quantifiers with the same constant whenever possible.

We ran experiments with three different datasets: easy, medium and hard. Each dataset includes five formulas ranging from relatively simple propositional formulas to more complex nested quantifier formulas. Specifically, the easy dataset includes only single-quantifier propositional formulas, and the medium/hard datasets include formulas with nested quantifiers and greater logical complexity.

Table 2 shows the experimental results. The improved solver performed significantly better than the baseline solver in terms of number of successfully proved formulas. Most notably, the improved solver achieved much higher success rates for the medium and hard datasets. E.g., on the hard dataset, the baseline solver only managed to prove one formula, while the improved solver managed to prove three formulas.

4 Discussion

This project shows how an automated reasoning system for first-order logic can be implemented using a sequent-calculus based approach. In comparison to the baseline solver, which proves many formulas efficiently but fails when encountering complex structures (such as implications) with multiple levels of quantification, the new solver has solved several times more formulas than the baseline solver in both medium and

hard data sets. However, the new solver does require slightly longer runtimes per formula than the baseline solver, although this additional cost is very low compared to its enhanced ability to find proofs.

Although the new solver can prove many formulas that were previously unprovable with the original solver, there are currently some limitations to the completeness of the proof search for all valid formulas. The new solver depends upon applying heuristically selected rules and restricting searches by depth. These restrictions likely limit the number of different ways that the solver explores each formula to find a proof. Also, although the new solver reduces the amount of time spent on creating unnecessary instances of variables through quantified terms, there are other optimizations that could potentially be made to improve performance.

Some examples of future work might include developing better heuristics for selecting rules used during the proof search, designing new forms of caching that would allow the solver to store fewer states for longer periods of time and thereby use less memory and/or perform faster, or using multi-threading to explore different potential proof-branches at the same time. Some examples of extending the capabilities of this solver might include adding support for more powerful logical systems and integrating with tools already being used for formal verification.

5 Data Availability

The source code for this project is available at: <https://github.com/gulatideepansh/FOL-solver.git>

References

- [1] Hou, Z.: Fundamentals of Logic and Computation: With Practical Automated Reasoning and Verification. Springer (2021)
- [2] Python Software Foundation: Python Language Reference. <https://www.python.org>